

How Databases Actually Store Your Data

Downloadable student notes for Week 3 Lesson 3



DB

By the end of this week, you should be able to:

- Explain why applications need databases instead of only variables or files.
- Describe tables, rows, columns, primary keys, and foreign keys.
- Recognize CRUD actions inside common LMS features.
- Sketch simple relationships between students, courses, progress, and quiz results.
- Explain why indexes, transactions, backups, and permissions matter.

Use these notes before the video, during class, and again before the quiz. Write your own examples in the margins or in a notebook.

Structured Lesson Notes

Read each section slowly. The goal is not memorizing vocabulary; the goal is explaining what happens inside a real application.

1. Why Apps Need Databases

Variables disappear when a program stops. Files can work for tiny projects, but databases give applications a reliable way to store, search, update, and protect information over time.

Concept	Meaning
Memory	Fast but temporary. It disappears when the program stops.
File	Simple for small data, but difficult to query safely as records grow.
Database	Designed for many records, users, searches, relationships, and rules.

```
$ sqlite3 school.db
CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT);
```

Try this: Name three pieces of data this LMS must keep after a student logs out.

2. Tables, Rows, Columns, and Keys

A relational database stores data in tables. Each row is one record. Each column is one field. A primary key uniquely identifies a row so the app can find the exact record again.

Concept	Meaning
Table	A collection of one kind of thing, such as students or quiz results.
Row	One saved item, such as one student.
Primary key	The unique ID for that item.

```
students
+----+-----+-----+
| id | name   | course |
+----+-----+-----+
| 1  | Zakariya | Computer |
```

Try this: Sketch a table for course enrollments with at least four columns.

3. CRUD: The Four Basic Data Actions

Most app features are variations of CRUD: create, read, update, and delete. A checkout, a quiz result, and a lesson progress update all fit this pattern.

Concept	Meaning
Create	Insert a new row.
Read	Find rows that match a question.
Update	Change an existing row.
Delete	Remove or archive data.

```
INSERT INTO students (name) VALUES ('Amina');  
SELECT * FROM students;  
UPDATE students SET course = 'Computer' WHERE id = 1;
```

Try this: For a student quiz submission, identify the create, read, and update actions.

4. Relationships

Real apps connect data. One student can have many quiz results. One course can contain many lessons. Relationships connect records without copying everything into every table.

Concept	Meaning
One-to-many	One course has many lessons, or one student has many quiz attempts.
Foreign key	A column that points to another table.
Join	A query that combines related tables.

```
students.id -> quiz_results.student_id  
courses.slug -> enrollments.course_slug
```

Try this: Draw the relationship between users, course enrollments, and lesson progress.

5. Indexes and Query Speed

An index is like the back of a textbook: it helps the database find rows faster. Indexes speed up reads, but they also take space and make writes a little more work.

Concept	Meaning
Without index	The database may scan many rows.
With index	The database can jump closer to the matching rows.
Tradeoff	Faster search, more storage and maintenance.

```
CREATE INDEX idx_students_username ON students(username);
```

Try this: Which columns in a login table would likely need an index?

6. Reliability and Protection

Databases need rules because users and programs make mistakes. Transactions keep groups of changes together, backups protect against loss, and permissions limit who can see or change data.

Concept	Meaning
Transaction	All steps succeed together or fail together.
Backup	A copy you can restore.
Permission	Who is allowed to read or write.

```
BEGIN;  
UPDATE accounts SET balance = balance - 500 WHERE id = 1;  
UPDATE accounts SET balance = balance + 500 WHERE id = 2;  
COMMIT;
```

Try this: Explain why checkout approval should not be saved only in the browser.

Mini Project

Design a small database plan for the LMS. Include tables for users, courses, enrollments, lesson progress, quiz results, and teacher resources. For each table, write the most important columns and explain one relationship.

Submission checklist:

- Use clear labels and short explanations.
- Show the main flow from user action to saved result.
- Include at least one mistake or failure case and how the system should respond.
- Submit the work through the class activity/project area so the teacher can review it.

Quick Review

- What data disappears when a program stops?
- What makes a primary key different from a normal column?
- Which CRUD action happens when a student submits a quiz?
- Why should enrollments connect users and courses instead of copying course data into every user row?
- What can go wrong if a checkout update is not protected by a transaction?

Study habit: After each topic, write one example from the LMS or a real app you use. That is how the vocabulary becomes practical.